

DxAssist - changelog prototypu

Stan dokumentu: 22 czerwca 2026 r.

Cel obecnego prototypu

Obecny prototyp DxAssist pokazuje pełny przepływ od zalogowania użytkownika, przez wybór modułu diagnostycznego i przekazanie danych, aż po otrzymanie przykładowego wyniku analizy.

Najważniejszym celem tej wersji jest potwierdzenie, że części systemu potrafią ze sobą współpracować:

- interfejs użytkownika pozwala lekarzowi uruchomić analizę,
- backend przyjmuje żądanie i pilnuje dostępu tylko dla zalogowanych użytkowników,
- scheduler przekazuje dane do właściwych modułów,
- moduły diagnostyczne zwracają wyniki,
- wynik wraca do użytkownika w czytelnej formie.

To nadal jest prototyp techniczny. Nie jest to gotowy produkt medyczny ani system przeznaczony do pracy z rzeczywistymi danymi pacjentów.

Co zostało zrobione

1. Uruchamialny lokalny system

Przygotowano lokalny zestaw usług uruchamiany przez Docker Compose. W aktualnej konfiguracji obejmuje on:

- aplikację frontendową dla użytkownika,
- backend API,
- bazę PostgreSQL,
- scheduler, czyli usługę koordynującą analizy,
- przykładowy moduł angiografii,
- przykładowy moduł badań krwi.

Dzięki temu można uruchomić cały prototyp jako kilka współpracujących kontenerów i sprawdzić przepływ danych między nimi.

2. Logowanie i kontrola dostępu

Dodano podstawowy mechanizm logowania użytkowników. Użytkownik może:

- zalogować się adresem e-mail i hasłem,
- korzystać z chronionych ekranów po zalogowaniu,
- automatycznie odświeżać sesję,
- wylogować się z systemu,
- pobrać i zaktualizować podstawowe dane własnego profilu.

W prototypie nie ma publicznej rejestracji. Konta użytkowników są tworzone administracyjnie, zgodnie z założeniem, że system ma być używany w środowisku kontrolowanym, np. w placówce medycznej.

3. Podstawowe zarządzanie użytkownikami

Backend posiada administracyjny mechanizm zarządzania użytkownikami. Administrator może tworzyć, przeglądać, edytować i usuwać konta.

Na tym etapie jest to podstawowy mechanizm administracyjny. Nie ma jeszcze rozbudowanego modelu ról, ścieżek akceptacji, historii zmian ani szczegółowych uprawnień dla różnych grup personelu.

4. Ekran diagnostyczny dla użytkownika

Frontend zawiera ekran, na którym zalogowany użytkownik może:

- zobaczyć dostępne moduły diagnostyczne,
- wybrać typ analizy,
- dodać pliki wejściowe,
- uruchomić analizę,
- zobaczyć wynik w formie raportu.

Pliki są obecnie zamieniane po stronie przeglądarki na postać tekstową i przekazywane dalej do backendu. Backend nie zapisuje tych plików na dysku.

5. Lista dostępnych analiz

Backend zwraca obecnie trzy pozycje diagnostyczne:

- `dxassist-angiography` - analiza obrazu angiografii,
- `dxassist-screening` - analiza danych z badań krwi,
- `dxassist-heartdisease` - analiza łączona, wykorzystująca angiografię oraz badania krwi.

Lista jest obecnie wpisana na stałe w backendzie. Nie jest jeszcze pobierana z centralnego rejestru modułów.

6. Połączenie backendu ze schedulerem

Najważniejszą zmianą prototypową jest działające połączenie między backendem a schedulerem.

Z perspektywy użytkownika wygląda to prosto: użytkownik uruchamia analizę i czeka na wynik. W tle backend przekazuje żądanie do schedulera, scheduler kontaktuje się z odpowiednim modułem diagnostycznym, a wynik wraca tą samą ścieżką do interfejsu użytkownika.

Backend obsługuje też sytuacje błędne, np. brak połączenia ze schedulerem, przekroczenie czasu oczekiwania albo niepoprawną odpowiedź.

7. Obsługa analizy pojedynczej

Dla pojedynczego modułu scheduler przekazuje dane do jednego modelu i zwraca jego odpowiedź.

W obecnej wersji dostępne są dwa przykładowe moduły pojedyncze:

- angiografia,
- badania krwi.

Są to moduły pokazowe. Zwracają stałe, przykładowe wyniki i nie wykonują rzeczywistej analizy

medycznej.

8. Obsługa analizy łączonej

Dodano prototyp analizy łączonej dla chorób serca. Taki scenariusz wykorzystuje więcej niż jeden moduł diagnostyczny.

Obecny przepływ wygląda następująco:

1. Scheduler uruchamia pierwszy moduł, czyli angiografię.
2. Po otrzymaniu wyniku prosi backend o dane dla kolejnego modułu.
3. Backend przekazuje dane do badań krwi, jeśli zostały wcześniej dostarczone przez frontend.
4. Scheduler uruchamia drugi moduł.
5. Scheduler łączy wyniki według ustawionych wag.
6. Backend zwraca wynik końcowy do frontendu.

W aktualnej konfiguracji analiza łączona używa wag:

- angiografia: 40%,
- badania krwi: 60%.

Łączenie wyników jest obecnie uproszczone. Scheduler szuka takich samych pól liczbowych w odpowiedziach modułów i wylicza z nich średnią ważoną. To wystarcza do pokazania koncepcji, ale nie jest docelowym algorytmem klinicznym.

9. Przykładowe moduły diagnostyczne

W repozytorium znajdują się dwa proste moduły pokazowe:

- moduł angiografii zwraca przykładowe prawdopodobieństwo choroby wieńcowej,
- moduł badań krwi zwraca przykładowe prawdopodobieństwo oraz przykładowe oznaczenia podwyższonych parametrów.

Te moduły służą do demonstracji komunikacji między usługami. Nie analizują rzeczywistych obrazów ani rzeczywistych wyników badań.

10. Raport z wynikiem

Frontend potrafi pokazać wynik analizy w formie raportu:

- główny wynik liczbowy,
- paski ryzyka dla parametrów liczbowych,
- szczegóły wyników cząstkowych przy analizie łączonej,
- informację o wagach modułów,
- komunikat przypominający, że wynik wymaga weryfikacji klinicznej.

Raport jest demonstracyjny. Nie ma jeszcze pełnego, zatwierdzonego medycznie formatu raportowania.

11. Podstawowa dokumentacja integracyjna

Dodano dokumentację opisującą:

- rolę backendu w prototypie,

- sposób komunikacji backendu z frontendem,
- sposób komunikacji backendu ze schedulerem,
- protokół komunikacji schedulera,
- lokalne uruchomienie systemu przez Docker Compose,
- znane ograniczenia wersji prototypowej.

Obecne możliwości prototypu

Na obecnym etapie prototyp pozwala zademonstrować:

- logowanie użytkownika,
- ochronę ekranów wymagających zalogowania,
- pobranie listy dostępnych analiz,
- przesłanie danych do analizy,
- uruchomienie analizy pojedynczej,
- uruchomienie analizy łączonej z użyciem dwóch modułów,
- przekazanie wyników z modułów do schedulera,
- połączenie wyników przez scheduler,
- zwrócenie wyniku do backendu,
- wyświetlenie raportu w aplikacji frontendowej,
- obsługę części typowych błędów komunikacji.

Prototyp pokazuje więc docelową ideę DxAssist: jeden system może pośredniczyć między użytkownikiem a wieloma wyspecjalizowanymi modułami diagnostycznymi.

Najważniejsze ograniczenia

1. Brak rzeczywistych modeli medycznych

Aktualne moduły diagnostyczne są atrapami. Zwracają stałe przykładowe wartości, niezależnie od przekazanych danych.

Oznacza to, że wyniki widoczne w aplikacji nie mają wartości medycznej i nie powinny być interpretowane jako analiza pacjenta.

2. Brak przechowywania analiz

Backend nie zapisuje obecnie:

- przesłanych danych,
- historii analiz,
- wyników analiz,
- raportów,
- informacji o przebiegu zapytania.

Po zakończeniu żądania wynik jest zwracany do użytkownika, ale nie jest utrwalany jako historia sprawy.

3. Brak audytu medycznego

Nie ma jeszcze dziennika audytowego pokazującego:

- kto uruchomił analizę,
- kiedy została uruchomiona,
- jakie moduły zostały użyte,
- jaka wersja modułu zwróciła wynik,
- kto oglądał wynik,
- czy wynik został zaakceptowany lub odrzucony.

Taki audyt będzie konieczny przed realnym użyciem w środowisku medycznym.

4. Brak docelowego zarządzania modułami

Lista modułów jest obecnie statyczna. System nie ma jeszcze panelu ani rejestru, w którym administrator może:

- dodawać nowe moduły,
- wyłączać moduły,
- sprawdzać ich stan,
- widzieć wersje modeli,
- zarządzać konfiguracją wag lub reguł.

5. Uproszczony scheduler

Scheduler działa jako techniczny koordynator, ale jego logika jest jeszcze bardzo prosta.

W szczególności:

- nie wybiera dynamicznie najlepszych modułów dla danego przypadku,
- nie ocenia jakości danych wejściowych,
- nie zna pełnego kontekstu klinicznego,
- nie obsługuje zaawansowanych scenariuszy awarii modułów,
- stosuje uproszczone łączenie wspólnych wartości liczbowych.

Obecna wersja potwierdza możliwość orkiestracji modułów, ale nie jest docelowym mechanizmem decyzyjnym.

6. Brak zaawansowanej walidacji danych

System sprawdza podstawowo, czy żądanie ma poprawną strukturę, ale nie waliduje jeszcze medycznej poprawności danych.

Nie ma jeszcze np. kontroli:

- jakości obrazu,
- formatu plików medycznych,
- kompletności badań,
- jednostek laboratoryjnych,
- zakresów referencyjnych,
- zgodności danych z konkretnym scenariuszem klinicznym.

7. Brak obsługi plików jako pełnoprawnych załączników

Frontend potrafi przyjąć plik i przekazać go dalej, ale backend nie ma jeszcze docelowego systemu uploadu plików.

Nie ma obecnie:

- magazynu plików,
- skanowania plików,
- kontroli typu i rozmiaru,
- polityki retencji,
- szyfrowanego przechowywania załączników.

8. Analizy są synchroniczne

Użytkownik czeka na wynik w ramach jednego żądania. Nie ma jeszcze kolejki zadań, pracy w tle ani śledzenia postępu.

Dla prawdziwych modeli, które mogą działać dłużej, potrzebny będzie mechanizm zadań asynchronicznych, statusów i powiadomień o zakończeniu.

9. Ograniczony model uprawnień

System rozróżnia użytkownika zalogowanego oraz administratora, ale nie ma jeszcze pełnego modelu ról klinicznych.

Nie ma jeszcze osobnych uprawnień dla np. lekarza prowadzącego, konsultanta, administratora technicznego, audytora czy opiekuna modułów.

10. Brak gotowości produkcyjnej

Prototyp nie jest gotowy do wdrożenia produkcyjnego. Brakuje m.in.:

- pełnego monitoringu,
- alertów,
- kopii zapasowych i procedur odtwarzania,
- twardych limitów zasobów,
- certyfikowanego bezpieczeństwa dla danych medycznych,
- szyfrowania i polityk retencji danych,
- testów obciążeniowych,
- procesu zatwierdzania modeli,
- dokumentacji operacyjnej dla placówki medycznej.

Znane niespójności i uwagi

Część dokumentacji głównej projektu opisuje scheduler i moduły jako elementy planowane. Aktualna gałąź prototypowa zawiera już działający scheduler oraz dwa pokazowe moduły diagnostyczne, dlatego stan kodu jest w tym obszarze nowszy niż część ogólnej dokumentacji projektu.

Przykłady wyników w dokumentacji integracyjnej mają charakter poglądowy. W aktualnym kodzie wartości zwracane przez moduły pokazowe są stałe i mogą różnić się od starszych przykładów tekstowych.

Podsumowanie dla klienta

Aktualny prototyp pokazuje, że architektura DxAssist może działać jako system pośredniczący między lekarzem a wieloma modułami diagnostycznymi. Działa logowanie, wybór analizy, przekazanie

danych, komunikacja z modułami przez scheduler oraz prezentacja wyniku.

Największą wartością tej wersji jest potwierdzenie pełnego przepływu od użytkownika do modułów i z powrotem. Największym ograniczeniem jest to, że warstwa medyczna jest jeszcze demonstracyjna: moduły są atrapami, logika łączenia wyników jest uproszczona, a system nie przechowuje historii ani nie posiada audytu wymaganego dla rzeczywistego środowiska klinicznego.

Kolejny etap powinien skupić się na zastąpieniu modułów demonstracyjnych realnymi modułami, dodaniu trwałej historii analiz, audytu, rejestru modułów, wersjonowania modeli oraz bezpiecznej obsługi danych medycznych.